Real-world (score an answer that isn't exact) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `llm_output_testing_practice.py` (Real-world section): this is the Build Task's `judge.py` in miniature — the grading rules written first, a second model call that checks the real output against them, and a calibration test proving the judge itself gets obvious cases right before you trust its verdict on anything subtle. One small task (summarizing a support ticket) keeps the focus purely on the judge, not on a whole pipeline. **If not:** the Build Task's judge would be the first judge you ever wrote, scoring Project 4's output with nothing to show it can actually be trusted.

The solution calls the real OpenAI API (`OPENAI_API_KEY` in your `.env`), so run it from inside `practice/` with `pytest llm_output_testing_practice.py -v`.

Solution

Story: "under 60 words" is a rule plain Python can check for free with `len(text.split())`, so it shouldn't cost a paid judge call at all — the judge is reserved for the rules that genuinely need judgment about meaning. A typed dataclass result plus a saved prompt version string make every score both readable and traceable back to exactly what produced it. **If not:** you'd pay for a model call just to count words, and a score saved last month couldn't be tied back to the judge prompt version that produced it.

```
# practice/llm_output_testing_practice.py — Real-world section
from dataclasses import dataclass
from openai import OpenAI

client = OpenAI()

TICKET = (
    "Order #4411 arrived 6 days late because of a warehouse delay. "
    "The customer wants a refund of the shipping fee."
)
SUMMARY_TASK = "Summarize this support ticket in 1-2 sentences:\n" + TICKET
JUDGE_RULES = [
    "mentions that the delivery was late",
    "mentions the shipping-fee refund request",
    "does not state any fact that is not in the ticket",
```

```
]
# why: every score is tied to one exact judge prompt — change the
# prompt, change this name
JUDGE_PROMPT_VERSION = "v1"

@dataclass
class JudgeResult:
    verdict: str
    reason: str

def call_model(prompt: str) -> str:
    # how: temperature=0 — a judge should be steady, not creative
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def build_judge_prompt(task: str, answer: str, rules: list[str]) -> str:
    rules_text = ""
    for rule in rules:
        rules_text = rules_text + "- " + rule + "\n"
    return (
        "Task: " + task + "\n"
        "Answer to check: " + answer + "\n"
        "Rules the answer must follow:\n" + rules_text +
        "Reply with PASS or FAIL on the first line, "
        "then one sentence explaining why."
    )

def score_output(task: str, answer: str, rules: list[str]) -> JudgeResult:
    prompt = build_judge_prompt(task, answer, rules)
    reply = call_model(prompt)
    # how: split once — first line is the verdict, the rest the reason
    lines = reply.strip().split("\n", 1)
    verdict = lines[0].strip().upper()
    if len(lines) > 1:
        reason = lines[1].strip()
    else:
        reason = ""
    return JudgeResult(verdict=verdict, reason=reason)

def test_summary_passes_judge():
    summary = call_model(SUMMARY_TASK)
    # when: a rule code can check gets checked by code, before the judge
    assert len(summary.split()) < 60
```

```
    result = score_output(SUMMARY_TASK, summary, JUDGE_RULES)
    assert result.verdict == "PASS", result.reason
```

Expected output:

```
llm_output_testing_practice.py::test_summary_passes_judge PASSED
```